

## 对比式图文预训练模型 CLIP

CLIP 的论文全名是 **Learning Transferable Visual Models From Natural Language Supervision**，由 OpenAI 在 2021 年提出。CLIP 是用大量图文对训练模型，让图片和对应文本在同一个向量空间中靠近，不对应的图片和文本远离。原论文摘要中提到，CLIP 使用了约 4 亿个互联网图文对进行预训练，并且可以通过自然语言描述完成零样本迁移任务。（<https://arxiv.org/abs/2103.00020>）

CLIP 要解决的问题是“输入图片+若干文本描述→判断哪句文本最匹配图片”  
例如：

图片 x：一只奶龙坐在沙发上

文本 t1：a photo of a NaiLong

文本 t2：a photo of a CST

文本 t3：a photo of a Computer

CLIP 要学会图片 x 和 t1 最匹配

CLIP 有两个主要部分

| 部分            | 作用      |
|---------------|---------|
| Image Encoder | 把图片变成向量 |
| Text Encoder  | 把文本变成向量 |

记：

$f_{\theta}(x)$  表示图像编码器，其中  $x$  是图片， $\theta$  是图像编码器的参数。

$g_{\phi}(t)$  表示文本编码器，其中  $t$  是文本， $\phi$  是文本编码器的参数。

于是  $v = f_{\theta}(x)$  是图片向量； $u = g_{\phi}(t)$  是文本向量。

CLIP 的目标是让  $v$  和  $u$  处在同一个向量空间里。

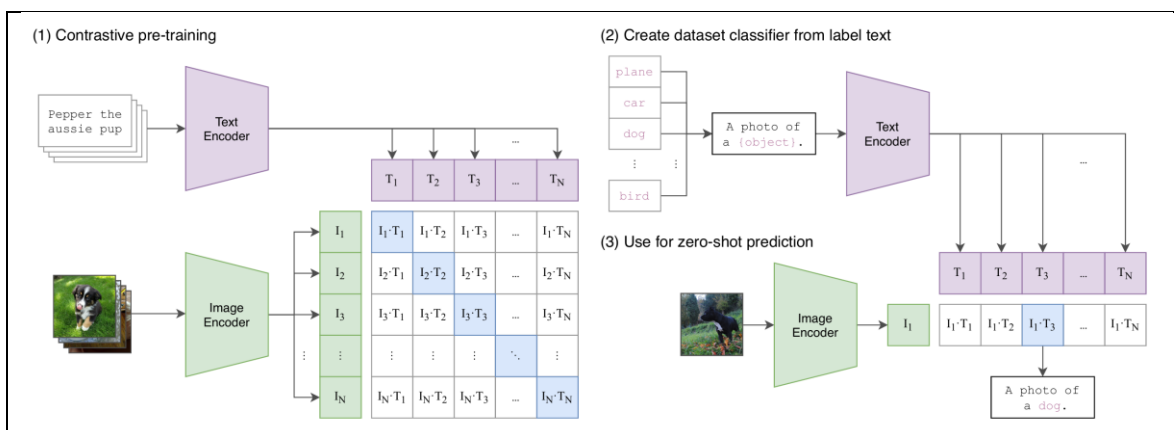
也就是“图片→向量；文本→向量”，然后用向量相似度判断它们是否匹配。OpenAI 官方仓库的 README（<https://github.com/openai/CLIP>）也说明，CLIP 提供 `model.encode_image(image)` 和 `model.encode_text(text)` 来分别得到图像特征和文本特征。

假设图片编码器输出： $v \in \mathbb{R}^d$ ；文本编码器输出： $u \in \mathbb{R}^d$ 。

这里的  $d$  是向量维度，例如 512、768 等。CLIP 通常会把向量归一化：

$$\hat{v} = \frac{v}{|v|}; \hat{u} = \frac{u}{|u|}$$

其中  $|v| = \sqrt{v_1^2 + v_2^2 + \dots + v_d^2}$ ，归一化之后  $|\hat{v}| = 1; |\hat{u}| = 1$ ，然后计算点积  $s = \hat{v}^T \hat{u}$ ，因为两个向量都已经归一化，所以  $\hat{v}^T \hat{u} = \cos(\hat{v}, \hat{u})$ ，CLIP 用归一化后的点积来表示图片和文本的语义相似度。官方仓库也说明，CLIP 的 forward 输出的是图像和文本之间的 logit 分数，其值来自对应图像特征和文本特征的余弦相似度。



[举例:

假设图片向量是  $\hat{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ; 文本 1 的向量是  $\hat{u}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ; 文本 2 的向量是  $\hat{u}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ 。

那么图片和文本 1 的相似度为  $\hat{v}^\top \hat{u}_1 = [1 \ 0] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$ ; 图片和文本 2 的相似度为  $\hat{v}^\top \hat{u}_2 = [1 \ 0] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0$ 。  $\hat{v}^\top \hat{u}_1 > \hat{v}^\top \hat{u}_2$ , 因此 CLIP 会认为图片更匹配文本 1。]

训练时, CLIP 一次输入一批数据。假设一个 batch 有  $N$  个图文对:

$$(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)$$

其中  $x_i$  表示第  $i$  张图片,  $t_i$  表示第  $i$  句文本。也就是第 1 张图片匹配第 1 句文本; 第 2 张图片匹配第 2 句文本; .....; 第  $N$  张图片匹配第  $N$  句文本。经过两个编码器

$v_i = f_\theta(x_i); u_i = g_\phi(t_i)$  归一化后  $\hat{v}_i = \frac{v_i}{|v_i|}; \hat{u}_i = \frac{u_i}{|u_i|}$ 。

把所有图片向量排成一个矩阵:  $V = \begin{bmatrix} \hat{v}_1^\top \\ \hat{v}_2^\top \\ \vdots \\ \hat{v}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}$

把所有文本向量排成一个矩阵:  $U = \begin{bmatrix} \hat{u}_1^\top \\ \hat{u}_2^\top \\ \vdots \\ \hat{u}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}$

那么所有图片和所有文本的相似度矩阵是  $S = VU^\top$ , 其中  $S \in \mathbb{R}^{N \times N}$ , 第  $i$  行第  $j$  列的元素是  $S_{ij} = \hat{v}_i^\top \hat{u}_j$ , 即第  $i$  张图片与第  $j$  句文本的相似度。

[例如 batch 里面有 3 个图文对, 那么相似度矩阵是  $S = \begin{bmatrix} S_{11} & S_{12} & S_{13} \\ S_{21} & S_{22} & S_{23} \\ S_{31} & S_{32} & S_{33} \end{bmatrix}$

理想情况下, CLIP 希望对角线元素  $S_{11}, S_{22}, S_{33}$  最大。]

对于第  $i$  张图片  $x_i$  来说, 正确文本是  $t_i$ , 错误文本是  $t_j$ ,  $j \neq i$ 。CLIP 的训练目标是  $S_{ii} \uparrow$  同时  $S_{ij} \downarrow$ ,  $j \neq i$ , 也就是正确图文对相似度变大; 错误图文对相似度变小。

现在我们有一张图片  $x_i$ ，和所有文本  $t_1, t_2, \dots, t_N$  的相似度：  $S_{i1}, S_{i2}, \dots, S_{iN}$

我们希望把这些相似度变成概率  $P(t_j | x_i)$ ，即在给定图片  $x_i$  的条件下，文本  $t_j$  是正确文本的概率。

用 Softmax:  $P(t_j | x_i) = \frac{\exp(S_{ij}/\tau)}{\sum_{k=1}^N \exp(S_{ik}/\tau)}$ ，其中  $\tau$  叫温度系数。如果  $\tau$  较小，那么

Softmax 会更加偏向最大相似度。为了简化讲解，可以先取  $\tau = 1$ ，于是：

$$P(t_j | x_i) = \frac{\exp(S_{ij})}{\sum_{k=1}^N \exp(S_{ik})}$$

对于第  $i$  张图片，正确答案是第  $i$  句文本。所以我们希望  $P(t_i | x_i)$  尽可能大。使用负对数似然损失  $\mathcal{L}_i^{I \rightarrow T} = -\log P(t_i | x_i)$ ，代入 Softmax:  $\mathcal{L}_i^{I \rightarrow T} = -\log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^N \exp(S_{ij}/\tau)}$ ，

整理得到  $\mathcal{L}_i^{I \rightarrow T} = -\frac{S_{ii}}{\tau} + \log \sum_{j=1}^N \exp(S_{ij}/\tau)$ ，整个 batch 的图像到文本损失是：

$$\mathcal{L}^{I \rightarrow T} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i^{I \rightarrow T}$$

CLIP 不只做“给图片找文本”，还反过来做“给文本找图片”。

对于第  $i$  句文本  $t_i$ ，正确图片是  $x_i$ 。所以定义：  $P(x_j | t_i) = \frac{\exp(S_{ji}/\tau)}{\sum_{k=1}^N \exp(S_{ki}/\tau)}$ ，注意

这里是看相似度矩阵的第  $i$  列。文本到图像的损失是  $\mathcal{L}_i^{T \rightarrow I} = -\log P(x_i | t_i)$ ，即：

$$\mathcal{L}_i^{T \rightarrow I} = -\log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^N \exp(S_{ji}/\tau)}$$

整个 batch 的文本到图像损失为：

$$\mathcal{L}^{T \rightarrow I} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i^{T \rightarrow I}$$

最终 CLIP 损失是两者平均为：

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}^{I \rightarrow T} + \mathcal{L}^{T \rightarrow I})$$

[举例：

假设 batch 里有 3 个图文对：

$x_1$ ：猫图片，  $t_1$ ：a photo of a cat

$x_2$ ：狗图片，  $t_2$ ：a photo of a dog

$x_3$ ：车图片，  $t_3$ ：a photo of a car

假设模型算出的相似度矩阵是  $S = \begin{bmatrix} 0.99 & 0.08 & 0.04 \\ 0.10 & 1.00 & 0.10 \\ 0.05 & 0.04 & 0.99 \end{bmatrix}$

现在先看第 1 张图片。它对应的相似度是：  $S_{11} = 0.99; S_{12} = 0.08; S_{13} = 0.04$

取  $\tau = 1$  则：  $P(t_1 | x_1) = \frac{e^{0.99}}{e^{0.99} + e^{0.08} + e^{0.04}}$

计算  $e^{0.99} \approx 2.6918; e^{0.08} \approx 1.0833; e^{0.04} \approx 1.0408$

所以  $P(t_1 | x_1) = \frac{2.6918}{2.6918 + 1.0833 + 1.0408} \approx \frac{2.6918}{4.8159} \approx 0.559$

损失  $\mathcal{L}_1^{I \rightarrow T} = -\log(0.559) \approx 0.582$

可以看出，即使正确相似度已经最大，只要 Softmax 后正确概率还不是很接近 1，损失仍然不是 0。

如果使用较小温度，例如  $\tau = 0.1$  那么：

$$P(t_1 | x_1) = \frac{e^{0.99/0.1}}{e^{0.99/0.1} + e^{0.08/0.1} + e^{0.04/0.1}} = \frac{e^{9.9}}{e^{9.9} + e^{0.8} + e^{0.4}}$$

这时正确项会占据更大比例。所以温度系数的作用是：**控制 Softmax 对相似度差异的敏感程度。**]

损失会拉近正确图文、推远错误图文。

先只看图像到文本损失：  $\mathcal{L}_i^{I \rightarrow T} = -\log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^N \exp(S_{ij}/\tau)}$

令  $p_{ij} = P(t_j | x_i) = \frac{\exp(S_{ij}/\tau)}{\sum_{k=1}^N \exp(S_{ik}/\tau)}$ ，则有一个重要结论  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ij}} = \frac{1}{\tau} (p_{ij} - \mathbf{1}_{j=i})$ ，

其中  $\mathbf{1}_{j=i} = \begin{cases} 1, & j=i \\ 0, & j \neq i \end{cases}$ 。

正确文本  $j=i$ ，此时  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ii}} = \frac{1}{\tau} (p_{ii} - 1)$ ，因为  $0 < p_{ii} < 1$ ，所以  $p_{ii} - 1 < 0$ 。

因此  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ii}} < 0$ 。梯度下降会做  $S_{ii} \leftarrow S_{ii} - \eta \frac{\partial \mathcal{L}}{\partial S_{ii}}$ 。因为偏导是负数，所以  $S_{ii}$  会变大。

这说明正确图文对的相似度会被提高。

错误文本  $j \neq i$ ，此时  $\mathbf{1}_{j=i} = 0$ ，所以  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ij}} = \frac{p_{ij}}{\tau}$ ，因为  $p_{ij} > 0$ ，所以  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ij}} > 0$ 。

梯度下降： $S_{ij} \leftarrow S_{ij} - \eta \frac{\partial \mathcal{L}}{\partial S_{ij}}$ 。所以  $S_{ij}$  会变小。这说明错误图文对的相似度会被降低。

因为  $S_{ij} = \hat{v}_i^\top \hat{u}_j$ ，为了简化推导，先忽略归一化的导数，直接看  $S_{ij} = v_i^\top u_j$

那么  $\frac{\partial S_{ij}}{\partial v_i} = u_j$ ，根据链式法则  $\frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial v_i} = \sum_{j=1}^N \frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial S_{ij}} \frac{\partial S_{ij}}{\partial v_i}$

$$\text{代入 } \frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial v_i} = \sum_{j=1}^N \frac{1}{\tau} (p_{ij} - \mathbf{1}_{j=i}) u_j, \text{ 拆开 } \frac{\partial \mathcal{L}_i^{I \rightarrow T}}{\partial v_i} = \frac{1}{\tau} \left[ (p_{ii} - 1) u_i + \sum_{j \neq i} p_{ij} u_j \right]$$

由于  $p_{ii} - 1 < 0$ ，所以正确文本向量  $u_i$  前面的系数是负的。

梯度下降  $v_i \leftarrow v_i - \eta \frac{\partial \mathcal{L}}{\partial v_i}$  会让  $v_i$  朝着  $u_i$  的方向靠近。

而错误文本向量  $u_j$  前面的系数是正的，梯度下降会让  $v_i$  远离这些错误文本方向。  
即  $v_i \rightarrow u_i; v_i \rightarrow u_j, j \neq i$ 。

前面只是说向量  $v_i$  和  $u_i$  如何变化。但真正训练时，更新的是编码器参数  $\theta$  和  $\phi$ 。因为  $v_i = f_\theta(x_i); u_i = g_\phi(t_i)$ ，所以损失函数本质上是  $\mathcal{L}(\theta, \phi)$ ，训练目标是  $\min_{\theta, \phi} \mathcal{L}(\theta, \phi)$ ，

使用梯度下降： $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}; \phi \leftarrow \phi - \eta \frac{\partial \mathcal{L}}{\partial \phi}$ ，其中  $\eta$  是学习率。

从链式法则看  $\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial v_i} \frac{\partial v_i}{\partial \theta}$ ，而  $v_i = f_\theta(x_i)$ ，所以  $\frac{\partial v_i}{\partial \theta}$  就是图像编码器内部的反向传播。同理  $\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial \phi}$

传统分类模型需要训练一个分类器  $x \rightarrow y$ ，其中  $y \in \{\text{cat}, \text{dog}, \text{car}, \text{bird}\}$ 。CLIP 不需要重新训练分类头。它把类别名称改写成文本 prompt:

$t_1 = \text{"a photo of a cat"}$   
 $t_2 = \text{"a photo of a dog"}$   
 $t_3 = \text{"a photo of a car"}$   
 $t_4 = \text{"a photo of a bird"}$

然后计算  $v = f_\theta(x)$  和  $u_1 = g_\phi(t_1), u_2 = g_\phi(t_2), u_3 = g_\phi(t_3), u_4 = g_\phi(t_4)$ ，归一化后，计算相似度： $s_1 = v^\top u_1; s_2 = v^\top u_2; s_3 = v^\top u_3; s_4 = v^\top u_4$ ，最后选择最大者  $\hat{y} = \arg \max_c s_c$ ，如果  $s_1$  最大，则模型预测 cat。

OpenAI 官方介绍中举过类似例子：如果任务是区分猫和狗，可以检查 CLIP 认为图片更可能匹配“a photo of a dog”还是“a photo of a cat”。

普通图像分类模型学的是  $F(x) = \begin{bmatrix} P(y = \text{cat} | x) \\ P(y = \text{dog} | x) \\ P(y = \text{car} | x) \end{bmatrix}$ ，输出类别是固定的。

而 CLIP 学的是两个函数  $f_\theta(x)$  和  $g_\phi(t)$ 。然后通过  $f_\theta(x)^\top g_\phi(t)$  判断图文是否匹配。因此 CLIP 的类别空间不是固定标签，而是自然语言。即：

**普通分类器：** 图片  $\rightarrow$  固定类别

**CLIP：** 图片+文本描述  $\rightarrow$  匹配程度

综上，CLIP 可以压缩成四个公式。

1. 编码：

$$v_i = f_\theta(x_i)$$

$$u_i = g_\phi(t_i)$$

2. 归一化：

$$\hat{v}_i = \frac{v_i}{|v_i|}, \quad \hat{u}_i = \frac{u_i}{|u_i|}$$

3. 相似度矩阵：

$$S_{ij} = \hat{v}_i^\top \hat{u}_j$$

4. 双向对比损失：

$$\mathcal{L} = \frac{1}{2} \left[ -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^N \exp(S_{ij}/\tau)} - \frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^N \exp(S_{ji}/\tau)} \right]$$